指令系统与 C 程序的机器级表示 4

章节脉络



本章围绕 “指令格式与编码 → 数据寻址 → 指令寻址与访问 → ISA 设计取舍 → 程序工具链 → 控制流与过程调用” 展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

不变量·操作码与操作数字段分开数

固定总长下，地址字段占用越多，可留给操作码的位越少。扩展操作码逐层使用未占用前缀，不是直接从全部位串数里减去指令条数。地址字段组合也占编码空间，容量必须按同一层级计算。

4.1 指令字段怎样决定可表示的操作和地址

定义 4.1.1 (概念与适用对象)

一条机器指令包含操作码以及完成该操作所需的其他字段。操作码规定做什么，地址字段可能给出立即数、寄存器编号、形式地址、位移或寻址方式，并非所有字段都是主存绝对地址。指令字长是指令编码的位数，机器字长是整数运算的基本位宽，两者不必相等。

按显式操作数地址字段数，可以有零地址、一地址、二地址、三地址指令。零地址指令也可以操作数据：栈式机器的加法隐含从栈顶取两个数；一地址指令可隐含使用累加器；二地址形式常让一个地址同时承担源和目的，但具体语义仍要由题目给出。“两个字段” 并不对应固定的访存次数：字段还可能指定寄存器或立即数。

数据传送指令移动或复制值；算术逻辑指令形成新值和可能的标志；转移指令改变后继执行地址；调用在转移前保留返回位置，返回根据保存的位置继续；输入输出及系统控制指令还可能有权限限制。通常的赋值和算术指令不会把源操作数 “取走后清空”。

固定长度操作码用 k 位可区分至多 2^k 种编码。扩展操作码把部分短编码保留为扩展前缀，用更长的操作码表示操作数较少的指令。已作为短指令使用的编码不能同时作长指令前缀，否则译码有歧义。

4.1.1 例 4-1 扩展操作码应逐层扣除哪些前缀

例 4.1.1 (配套例题)

【配套例题】设计某指令系统时，假设采用 16 位定长指令字格式，操作码使用扩展编码方式，地址码为 6 位，包含零地址、一地址和二地址 3 种格式的指令。若二地址指令有 12 条，一地址指令有 254 条，则零地址指令的条数最多为（ ）。

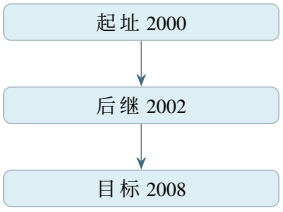
A. 0 B. 2 C. 64 D. 128

解析。二地址格式需要 12 位地址，短操作码为 4 位，共 16 种前缀，用掉 12 种，剩 4 种可扩展。每个前缀再接 6 位，共提供 $4 \times 64 = 256$ 种一地址操作码；用掉 254 种，剩 2 种再各扩展 6 位，得到 $2 \times 64 = 128$ 条零地址指令，选 **D**。

不能从 2^{16} 中直接减去 “12+254”，因为每条二地址指令还对应 2^{12} 种操作数字段组合，每条一地址指令对应 2^6 种组合。逐层保留前缀的算法直接反映译码过程，也方便处理其他字段长度。

最小反例·相对基准不一定是起点

最小反例：2B 指令从 2000H 取完，PC 已变 2002H，再加 6 应得 2008H。直接用指令起点加 6 得到 2006H，漏算了取指更新。负位移还须先按给定位宽作符号解释。



2B 指令取完后加 6，目标为 2008H。

4.2 从形式地址到有效地址

以下记指令地址字段为 A，寄存器 R 的内容为 (R)，主存地址 x 的内容为 M[x]；有效地址 EA 指最终操作数所在的主存地址。立即数不需要另求操作数的主存地址；寄存器寻址的操作数也在寄存器中，不必为凑访存次数而虚构一次主存访问。

寻址方式	操作数或有效地址	核对条件
立即	操作数在指令中	字段的符号扩展规则
直接	EA=A，操作数 =M[A]	A 的编码范围
一次间接	EA=M[A]	先取地址，再取操作数
寄存器直接	操作数 =(R)	字段是寄存器编号
寄存器间接	EA=(R)	寄存器保存的是地址
基址	EA=(BR)+ 位移	位移位宽及符号
变址	EA=A+(IX)	元素下标是否已乘元素宽度

寻址方式	操作数或有效地址	核对条件
相对	目标 =PC 基准 + 位移	PC 指当前还是后继地址

间接寻址得到的中间地址位宽可能大于指令地址字段，从而扩大可寻址范围，代价是多一次取地址的访存。基址可用于程序重定位，变址适合数组遍历；二者都可写成寄存器加位移的形式，但寄存器的用途不同，具体机器也可能用同一种编码机制实现。

4.2.1 例 4-2 相对转移用哪个 PC

例 4.2.1 (配套例题)

【配套例题】某机器字长为 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一个字节为操作码字段，第二个字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为 2000H，相对位移量字段的内容为 06H，则该转移指令成功转移后的目标地址是（ ）。

A. 2006H　B. 2007H　C. 2008H　D. 2009H

解析。指令从 2000H 开始，占 2 B，取完后 PC=2002H。相对位移为 +6，目标为 2008H，选 **C**。若位移字段改为 FAH 且按 8 位补码解释，则位移为 −6，目标为 1FFCH；不能把 FAH 当作正 250。PC 基准必须以本题规定的更新时机为准。

数组寻址还需区分元素下标和字节偏移。按字节编址、int 为 4 B 时，`a[i]` 地址为 base+4i。某 ISA 若有 “比例变址”，可由硬件把索引乘 4；若没有，则由移位或其他指令形成 4i。这不意味着数组元素之间多留了 3 个空洞，四个字节共同存放一个元素。

4.2.2 访存次数要按执行路径来数

若指令为一个字、无 Cache、无预取，直接寻址的读操作数指令通常至少取指一次、取操作数一次；一次主存间接寻址再多读一次地址。若还写回主存，需另计一次写。若指令跨两个存储字、发生缺页或含两个主存操作数，次数会改变。“寄存器间接” 中的 “间接” 不会额外读一次主存地址，因为地址已经在寄存器中。

不变量·只数真实读写事件

无 Cache 无预取时，取指、取有效地址、取操作数和写回主存各是独立事件。某字段在寄存器中就不应虚构一次主存读取；指令跨字或有两个主存源时也不能沿用一字一源的计数。

4.3 RISC 与 CISC：设计倾向不代替具体实现

RISC 通常采用较规整的指令格式、较少的寻址方式和 Load/Store 风格：算术逻辑操作主要在寄存器间进行，访存由装入和存储指令承担。这有利于规整译码和流水处理。CISC 往往支持更丰富的指令形式与寻址方式，单条指令可以表达较复杂操作。

下表按共同维度比较教材中的典型设计。表中的 “较多”“较少” 只是同一历史设计背景下的倾向，并不按指令条数、某条指令或单项性能为所有处理器划定边界。

比较维度	RISC 的典型设计	CISC 的典型设计	做题时的适用边界
指令功能与使用	选用常见的简单操作，复杂功能由指令组合实现	指令功能较丰富，一条指令可表达较复杂操作	不用 “小于 100 条／大于 200 条” 等历史概数代替具体 ISA 定义
编码与寻址	格式较规整，常用定长编码，寻址方式较少	格式和寻址方式较多，常见变长编码	定长、变长须按题给编码确认；不能只凭 RISC 名称确定每条指令字节数
数据操作数访存	典型 Load/Store 模型中，装入和存储访问数据存储器，算术逻辑在寄存器间进行	多类指令可含主存操作数	这里说的是数据访问，取指也要读存储器；具体指令能否访存仍查其语义
寄存器与调用	通用寄存器较多，多用寄存器传参，减少数据访存	传统设计的通用寄存器相对较少，操作数来源较丰富	寄存器数由 ISA 给定；传参位置还受 ABI 约束，不由 RISC/CISC 名称单独决定
执行时间与 CPI	简单操作便于规整流水；教材常用多数简单指令单周期的理想模型	不同指令的执行时间差异往往较大	单条指令延迟、流水吞吐和平均 CPI 分别计算；访存等待、乘除及发射宽度都会改变结果
控制方式	多采用硬布线控制	复杂指令常借助微程序控制	两者是控制器实现方式，可结合使用，不能断言某类 ISA 只能选一种
流水线	规整编码和操作有利于流水处理	也可通过译码、分解内部操作等方式实现流水	采用流水线不是 RISC 独占的能力；阶段数、旁路和阻塞规则均由具体微架构决定
编译优化	复杂功能较多依靠编译器选指令、分配寄存器和调度	丰富指令给出更多选择，选指令与优化可能更复杂	两类都需要编译优化；目标代码条数较少不直接等于执行时间较短

做题时要按题目中 “多采用”“通常” 等措辞判断典型特征；删去这些限定，就会把设计倾向误读成必然条件。

课程连接·ISA 不等于微架构

是否采用流水线、旁路和哪种控制器主要涉及微架构实现，ISA 规定软件可见指令语义。两类层次相互影响但不能互代。指令简单有利于规整实现，不意味着只可能采用一种硬件结构。

4.3.1 例 4-3　RISC 设计能否支持流水线

例 4.3.1 (配套例题)

【配套例题】下列关于 RISC 的叙述中，错误的是（　）。

A. 多采用硬连线方式实现控制器
B. 通常采用 Load/Store 型指令设计风格
C. 难以采用流水线数据通路实现微架构
D. 多采用寄存器传递过程调用时的参数

解析。格式规整、操作较简单，通常有利于流水线实现，所以 C 错。A、B、D 描述的是常见设计倾向，不应把 “多采用” 读成 “只能采用”。答案为 **C**。

4.4 编译、汇编、链接怎样确定程序中的地址

第 1 章已给出工具链顺序，这里进一步说明为什么目标文件未必能直接运行。编译和汇编时，一个模块可能引用另一个模块的函数或全局变

量，汇编器尚不知道它的最终地址，只能留下符号与重定位信息。链接器合并相应节，解析符号，再按选定地址修改相关引用。

可执行文件包含机器代码，也包含装入所需的组织信息；CPU 并不把文件的每一个字节都当作指令。目标文件还可能含符号表、调试信息和重定位记录。高级语言的变量名在运行时未必仍对应一个固定的主存地址；优化后变量本身可能在寄存器中、被折叠成常量，或已经消失。

错项排除·目标代码不是最终运行状态

汇编得到可重定位目标文件，还可能缺跨模块符号地址。链接并安排引用后形成可执行文件，运行还需装入和环境。把汇编与链接交换，既无法解析源语言，也无法解释地址何时确定。

4.4.1 例 4-4　工具链顺序

例 4.4.1 (配套例题)

【配套例题】将高级语言源程序转换为可执行目标文件的主要过程是（　）。

A. 预处理 → 编译 → 汇编 → 链接
B. 预处理 → 汇编 → 编译 → 链接
C. 预处理 → 编译 → 链接 → 汇编
D. 预处理 → 汇编 → 链接 → 编译

解析。先处理源文件中的预处理指令，再将高级语言翻译成汇编表示，继而编码为目标文件，最后解析跨模块引用，选 **A**。这是题目采用的典型静态编译流程；动态链接把部分工作留到装入或运行时，但汇编编码仍必须在程序开始执行前完成。

4.5 分支和循环怎样改变 PC

机器执行选择结构时，先形成条件，再决定是否改变后继地址。条件可由比较指令设置标志，也可由分支指令直接比较寄存器。翻译 C 条件时必须保留 signed/unsigned 的区别：两者都用减法硬件，但转移条件并不相同。

下列教学指令约定整数均为 32 位，R1 存放 x，R2 存放 y，`bge` 按有符号数比较，标签代表指令地址；并假定本例的差值可用 32 位有符号数表示：

```
    bge R1, R2, ELSE     // x>=y 时跳过 then
    sub R3, R2, R1      // then: z=y-x
    jump END
ELSE:
    sub R3, R1, R2      // else: z=x-y
END:
```

这里要实现 `if(x<y) z=y-x; else z=x-y;`，分支条件取原条件的否定，于是满足原条件时顺序落入 then。转移目标是代码位置，不是把标签的字符存进 PC。

分析循环时，先确定入口测试、循环体、更新和回边。对以下机器级教学轨迹，R0 为数组基址，R1 为元素下标，R2 为和，R3 为元素个数，int 占 4 B：

```
    R1 <- 0; R2 <- 0
TEST:
    if R1 >= R3 jump EXIT
    R4 <- R1 << 2
    R5 <- M[R0 + R4]
    R2 <- R2 + R5
    R1 <- R1 + 1
    jump TEST
EXIT:
```

不变量·返回地址属于调用点

调用保存的是调用之后继续执行的位置；函数入口只是被调目标。递归各层可以在同一代码位置调用，却有独立参数和栈帧。恢复现场时必须与当前返回层次配对，不能只追踪函数名。

R3=0 时直接退出，不读 `a[0]`；R3=3 时读取偏移 0、4、8 而非 0、1、2。这两个边界足以检查循环上下界和地址单位。若变换为指针遍历，每轮指针加 4，终止地址为 base+4R3；终止地址并不是最后一个有效元素的地址。

逆向读代码时，先标出各寄存器的初值和更新式，再识别分支条件。循环回边指令的执行次数与循环体次数可能不同：入口先测的 n 次循环，测试通常执行 n+1 次。算动态指令数和 Cache 访问次数时，这个差别必须计入。

4.6 过程调用为什么要保存返回位置

调用至少要解决四件事：把参数交给被调过程，记录返回位置，给局部变量和临时值安排空间，将返回值交回调用者。ISA 提供调用、返回或寄存器转移等基础机制，ABI 进一步规定参数寄存器、返回值位置、栈对齐，以及哪些寄存器由哪一方保存。

返回地址可以放到栈，也可以先放到链接寄存器。若采用链接寄存器，叶子函数可能不访存保存返回地址；会再次调用其他函数的过程，通常要在覆盖链接寄存器之前保存其中的返回地址。“返回地址一定写主存” 并不是所有指令系统的共同特征。

以下采用一个明确的栈模型说明调用：栈向低地址增长，每槽 4 B，SP 指向当前栈顶；push 先 SP 减 4 再写，pop 先读再 SP 加 4；调用压入后继指令地址，返回弹出该地址送 PC。调用前 SP=1000H，返回地址为 2004H，则调用后 SP=0FFCH、M[0FFCH]=2004H。

若被调过程又保存旧帧指针 FP、再分配 8 B 局部区，SP 依次变为 0FF8H 和 0FF0H。此时局部变量不能覆盖 0FFCH 的返回地址。退出时先撤销局部区，恢复旧 FP，再取返回地址；SP 最终回到 1000H。图中的地址单位为字节。

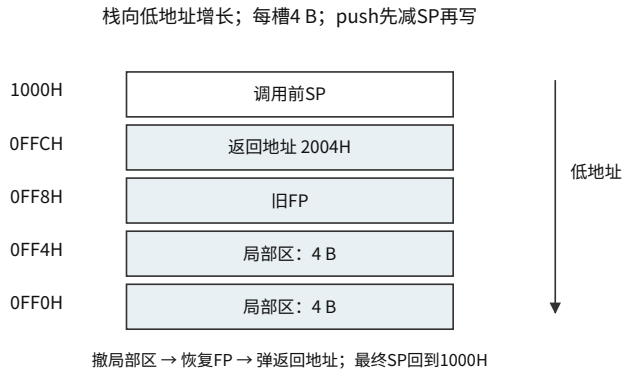


Figure 4.1: 调用、建立栈帧和返回时，SP 逐步移动

嵌套调用靠每次独立保存的返回位置区分 “回到哪里”；递归的各层也各有参数、局部变量和返回地址。局部变量同名不表示它们共用同一槽。调用者保存寄存器需在调用前保留自己还要使用的值；被调用者保存寄存器则在被调过程修改前保存、返回前恢复。

题目给出具体调用约定时，要按该约定追踪，不能凭熟悉的 x86、MIPS 或 RISC-V 经验补条件。特别要检查栈生长方向、SP 指向、入栈前后调整顺序和返回地址基准。这些细节都会影响地址计算与最终返回位置。

最小反例·转移不一定保存返回地址

最小反例：普通条件分支跳到标签，并不会自动为未来返回保留调用点。调用与分支都能改 PC，但保存返回位置的语义不同；把任意跳转都当成函数调用，会错误增加栈写和返回动作。

4.6.1 顺序、分支、调用和返回分别改变什么

执行方式	后继地址怎样得到	额外需要保留的信息	读代码时先核对什么
顺序执行	当前地址加指令字节数	不因普通顺序执行单独保存返回地址	变长指令必须实际数编码字节
条件分支	条件成立取目标，否则取顺序后继	通常无需调用返回地址	signed/unsigned 条件、位移符号、基准与缩放
调用	无条件转到被调入口	调用后的后继地址	返回位置存栈还是链接寄存器
返回	取先前保存的返回位置	还须按约定恢复栈和现场	返回地址来自哪层调用，不能重算成函数入口

相对寻址只规定 “基准地址加位移”，并不规定所有指令的位移都以字节为单位。例如下面 x86 的 call 用后继指令地址加有符号字节位移；某些定长 RISC 分支会把编码位移先左移两位。读机器码时应分别求出当前地址、指令长度、基准地址、扩展后的位移及缩放比例，最后核对目标。把汇编目标标签直接当成机器码里保存的绝对地址，会漏掉这条计算链。

4.6.2 例 4-5　递归调用的返回位置、机器编码与乘法溢出

例 4.6.1 (配套例题)

【配套例题】已知 $f(n)=n!=n\times(n-1)\times(n-2)\times\cdots\times2\times1$，计算 $f(n)$ 的 C 语言函数 f1 的源程序及其在 32 位计算机 M 上的部分机器级代码如下。

```
int f1(int n) {
    if (n > 1)
        return n * f1(n - 1);
    else
        return 1;
}
```

以下按 “行号／虚拟地址／机器指令（字节按地址递增排列）／汇编指令” 四列给出，每行是一条代码记录；省略行仍保留四列。

```
 1  00401000  55              push ebp
 …  …         …               …
11  00401018  83 7D 08 01     cmp dword ptr [ebp+8], 1
12  0040101C  7E 17           jle f1+35h (00401035)
13  0040101E  8B 45 08        mov eax, dword ptr [ebp+8]
14  00401021  83 E8 01        sub eax, 1
15  00401024  50              push eax
16  00401025  E8 D6 FF FF FF  call f1 (00401000)
 …  …         …               …
19  00401030  0F AF C1        imul eax, ecx
20  00401033  EB 05           jmp f1+3Ah (0040103A)
21  00401035  B8 01 00 00 00  mov eax, 1
 …  …         …               …
26  00401040  3B EC           cmp ebp, esp
 …  …         …               …
30  0040104A  C3              ret
```

其中，机器级代码行包括行号、虚拟地址、机器指令和汇编指令，计算机 M 按字节编址，int 型数据占 32 位。请回答下列问题。

（1）计算 f(10) 需要调用函数 f1 多少次？执行哪条指令会递归调用 f1？

（2）上述代码中，哪条指令是条件转移指令？哪几条指令一定会使程序跳转执行？

（3）根据第 16 行的 call 指令，第 17 行指令的虚拟地址应是多少？已知第 16 行的 call 指令采用相对寻址方式，该指令中的偏移量应是多少（给出计算过程）？已知第 16 行的 call 指令的后 4 字节为偏移量，M 采用大端方式还是小端方式？

（4）$f(13) = 6\,227\,020\,800$，但 f1(13) 的返回值为 1 932 053 504，为什么两者不相等？要使 f1(13) 能返回正确的结果，应如何修改 f1 的源程序？

（5）第 19 行的 imul 指令（带符号整数乘）的功能是 R[eax] ← R[eax] × R[ecx]，当乘法器输出的高、低 32 位乘积之间满足什么条件时，溢出标志 OF=1？要使 CPU 在发生溢出时转异常处理，编译器应在 imul 指令后加一条什么指令？

不变量·call 位移用后继地址校验

五字节 call 从 00401025H 开始，后继为 0040102AH；跳到 00401000H 所需位移为 −42，编码 FFFF FFD6H。把目标减起点会少算 5B。将求出的基准加位移回代，必须恰好恢复目标地址。

解析。先追踪源程序的调用层次，再读指令对 PC 和数据的作用。表中的省略号表示原题未展示的指令，行号差并不等于字节数。

（1）调用参数依次为 10、9、…、1，共 10 次，包含最外层调用；函数内部第 16 行 call 实际递归执行 9 次。参数为 1 时，第 12 行转到返回 1 的分支，不再递归。各层的返回位置虽都是调用点的后继地址，但栈帧和参数相互独立，返回时逐层乘回 2、3、…、10。

（2）第 12 行 jle 是条件转移，根据比较结果决定是否转移。第 16 行 call、第 20 行 jmp、第 30 行 ret 在执行时都会改变顺序后继：分别是调用、无条件跳转和返回。第 15 行 push 保存参数，本身并不转到 f1 入口。

（3）call 从 0040 1025H 开始，占 1 字节操作码及 4 字节位移，共 5 B，所以后继地址为

$$\texttt{00401025H} + 5 = \texttt{0040102AH}.$$

按后继地址为相对基准，位移为 $\texttt{00401000H} - \texttt{0040102AH} = -\texttt{2AH} = -42$，其 32 位补码为 FFFF FFD6H。表中后 4 字节按低地址到高地址为 D6、FF、FF、FF，低有效字节先存，因此为小端。复核：

`0040102AH` + (−42) = `00401000H`。不能用 call 自身起点减目标，也不能把 D6 当成正 214。

（4）13! 超过 32 位 int 上界 $2^{31}-1$。题给返回值是保留低 32 位后的机器结果：$6\,227\,020\,800 - 2^{32} = 1\,932\,053\,504$，它虽为正数，仍是溢出后的错误数学结果。标准 C 并没有规定有符号溢出必须这样回绕；这里解释的是原题给定的机器执行结果。

一种修改是把函数返回类型改为至少 64 位的 `long long`，使递归返回值及与 n 的乘法都按该较宽类型进行：

```
long long f1(int n) {
    if (n > 1)
        return n * f1(n - 1);
    return 1LL;
}
```

13! 可以用该类型精确表示。若只是先在 int 中乘完，再把溢出结果强制转换为宽类型，已经丢失的高位无法恢复。这里仅保证题目所问的 f1(13)，不意味着任意大的阶乘都能用固定 64 位整数表示。

（5）令精确 64 位乘积为高 32 位 H 与低 32 位 L。若 H 恰是 L 最高位的 32 位符号扩展，即 L 的符号位为 0 时 H 全 0、为 1 时 H 全 1，则截为 32 位仍保留真值，OF=0；否则 OF=1。等价地，乘积从第 63 位至第 31 位的高 33 位必须全 0 或全 1 才不溢出。只检查 H 是否为 0 并不够：可表示的负积需要 H 全 1。

题目要求的是发生溢出时转异常处理，因此应在 imul 后安排检查 OF 的**溢出自陷指令**：OF=1 时发出溢出异常并转入处理程序。普通条件转移 `jo` 能根据 OF 跳往一段代码，但跳转本身并不等于完成了异常入口动作。乘法器设置标志、指令检查标志、异常机制保存返回状态并转入口，是相接的三个步骤。

4.7 用语义、地址与状态三张清单读指令

机器级代码五步法

1. 列 ISA 给出的每条指令语义，区分它读寄存器、读主存、写主存还是只改 PC。
2. 标指令起址和字节长度，按题设确定顺序后继与相对基准。
3. 写有效地址，保留位移符号、索引比例和寻址级数。
4. 对分支列出真假两条路径；对调用单独记录返回地址、SP、FP 及需保存的值。
5. 用零次循环、一次循环、嵌套调用和负位移回代，最后数动态执行次数与访存。

4.7.1 扩展操作码为什么要保留前缀

译码互斥与编码空间

若某短操作码已被认作一条完整指令，同一前缀后的位就按其操作数字段解释；再把该前缀延长当作另一操作码，会让同一位串出现两种译码。因此扩展编码必须先留下未使用短前缀，再把它们的后续位作为更长操作码。每留下一个前缀并扩展 k 位，得到 2^k 个子前缀。逐层扣除已用子前缀，正是原例的容量算法。

不变量·编码数先核层级

扩展 k 位使一个保留前缀分裂成 2 的 k 次方个子前缀，已占用短格式对应的是整片子树。每一步统一数同样长度的前缀，才不会把指令条数与完整操作数字段位串混在一起。

□

课程连接·栈模型接到数据结构

栈的后进先出性质保持不变，但机器 SP 到底指向当前元素还是下个空位属于实现约定。数据结构中的 push 抽象不直接决定地址先减还是先写。题目给出约定后，再逐步画 SP 轨迹。

4.7.2 地址字段数不等于实际操作数数

隐含寻址不在指令中显式写出某个操作数的位置，例如约定累加器或栈顶。零地址算术指令仍可从栈顶取得两个操作数并把结果送回栈；只是这些位置由机器约定确定。一地址指令也可能隐含累加器，故题目问地址码个数时，不能直接回答参与运算的值有几个。

堆栈寻址以 SP 和约定确定元素位置。满栈/空栈、向低/高地址增长、先改 SP 还是先访存，都会改变读写地址。基址、变址和相对寻址虽然常都出现 “寄存器 + 位移” 的代数式，寄存器角色分别服务于重定位、数组下标与控制转移；代数相同不代表使用约定可互换。

4.7.3 指令类别与读写副作用

数据传送类把值在寄存器与存储器等位置间移动；算术逻辑类形成新值及可能的标志；移位类改变位位置；顺序控制类决定后继 PC；I/O 及系统控制类涉及端口、状态与受保护操作。具体指令是否改标志、访存或具有特权，须查给定语义，不能仅凭助记符字母猜测。

机器码解读和 C 语义需要两层核对。机器指令可以按规定生成截断结果，C 程序是否允许这种执行仍受溢出、对象范围与类型规则约束。把一次具体机器执行轨迹说清楚，并不等于证明对应 C 代码在所有实现上都有同样结果。

4.8 地址个数、指令长度与访存次数

地址字段数描述指令中显式给出的地址个数。三地址指令可分别给出两个源和一个结果地址；二地址指令常把一个源地址同时作为目的地址；一地址指令可隐含累加器；零地址指令可以不使用操作数，也可以隐含使用栈顶操作数。分析 “几个操作数” 时应包括隐含操作数，分析 “几个地址字段” 时只数编码字段。

若操作数和结果均在主存、指令一次取出，三地址运算可能需要取指 1 次、读源 2 次、写结果 1 次，共 4 次访存；寄存器操作数、间接寻址或多字指令都会改变次数。因此先列执行动作，再累计访问次数，不能把地址数直接当成访存次数。

指令字长、机器字长和存储字长分别描述指令编码、CPU 数据处理宽度和存储单元宽度。若存储字长 16 位，一条 32 位指令跨两个存储字，取指可能需要两次存储器访问；是否还能合并为一次总线事务，取决于具体存储组织。

4.8.1 扩展操作码的完整分配例

设 16 位指令，每个地址字段 4 位，要求 15 条三地址、12 条二地址、62 条一地址和 32 条零地址指令。基本 4 位操作码共有 16 种，先分配 15 种给三地址，剩余 1 种前缀扩展为 16 种 8 位操作码；分配 12 种给二地址，剩余 4 种再扩展为 64 种 12 位操作码；分配 62 种给一地址，剩余 2 种扩展为 32 种 16 位零地址操作码。

每次可继续扩展的数量为 “本层保留前缀数 $\times$ 下一段编码组合数”。已经分配的短操作码不能再作为其他操作码的前缀，否则译码器无法确定后续位是地址还是操作码。这里指令字长固定，变化的是操作码和

地址字段之间的分配。

课程连接·形式地址与有效地址

形式地址是指令字段，有效地址是计算后访问操作数的位置；立即寻址的字段本身表示操作数。

4.9 数据寻址方式的统一比较

下面把指令中的形式地址记为 A，寄存器内容记为 (R_i)，存储器单元内容记为 $M[A]$。执行阶段访存次数仅统计读取一个源操作数，取指和写回另计。

寻址方式	操作数或有效地址	执行阶段读主存次数
立即	操作数就是 A	0
直接	EA $=A$，操作数 $M[A]$	1
一次间接	EA $=M[A]$	2
寄存器	操作数为 (R_i)	0
寄存器间接	EA $=(R_i)$	1
相对	EA $=\mathrm{PC}_{\text{base}}+\mathrm{sext}(A)$	依操作数位置确定
基址	EA $=(\mathrm{BR})+A$	内存源通常 1 次

寻址方式	操作数或有效地址	执行阶段读主存次数
变址	EA $=(\mathrm{IX})+A$	内存源通常 1 次

相对寻址以题设规定的 PC 值为基准。常见机器用下一条指令地址作基准，位移为补码；若位移按字、半字对齐，还须先按约定缩放。基址寻址把一段程序或数据的起始位置交给基址寄存器，适合重定位；变址寻址把数组起点放在地址字段、元素变化量放在变址寄存器，适合循环访问。二者都表现为加法，寄存器含义和管理方式却不同。

堆栈寻址隐含使用 SP。采用向低地址增长、SP 指向栈顶元素的 32 位字栈时，压栈先使 SP 减 4 再写入，弹栈先读出再使 SP 加 4。若题目采用空栈顶指针或向高地址增长的约定，应重新写出操作次序。寄存器组组成的硬堆栈与主存组成的软堆栈在访存开销上不同。

4.10 x86 汇编：操作数顺序与宽度先行

课件采用 32 位 x86 示例，EAX、EBX、ECX、EDX 是 32 位通用寄存器；AX、AL、AH 分别对应 EAX 的低 16 位、最低 8 位和次低 8 位；ESP 和 EBP 常用于栈顶及栈帧基址。这里的具体寄存器约定属于示例 ISA，不能外推成所有机器的统一规则。

含义	Intel 格式	AT&T 格式
将立即数 5 送 EAX	`mov eax, 5`	`movl $5, %eax`
将 EBX 送 EAX	`mov eax, ebx`	`movl %ebx, %eax`
读地址 EBX+ECX×4+8 处 32 位数	`mov eax, dword ptr [ebx+ecx*4+8]`	`movl 8(%ebx,%ecx, 4), %eax`

Intel 格式目的在前、源在后；AT&T 格式源在前、目的在后，寄存器前有%，立即数前有 `$`，常用 b、w、l 后缀指定 8、16、32 位宽度。x86 的地址比例因子为 1、2、4 或 8，超出该集合的乘法不能直接当成这一寻址编码的比例因子。

`mov` 复制数据；`lea` 计算有效地址的算术表达式并把地址值送寄存器，不读取该地址中的数据；`add/sub` 修改目的数；`and/or/xor` 作逐位逻辑运算；`cmp` 计算减法标志而不保存差；`test` 计算与运算标志而

不保存按位与结果。移位时应区分 `shl/sal`、`shr` 和 `sar`，最后一个按算术右移保留符号扩展。

课程连接·调用边界

返回地址、保存的寄存器和局部变量各有职责；栈增长方向和参数位置必须采用题中调用约定。

4.10.1 条件转移如何解释相同标志位

以 Intel 格式 `cmp eax, ebx` 为例，标志来自 EAX−EBX。相等看 ZF；无符号小于看 CF；无符号小于等于看 CF 或 ZF；有符号小于看 SF $\neq$ OF；有符号小于等于看 ZF 或 SF $\neq$ OF。有符号大于要求 ZF=0 且 SF=OF，无符号大于要求 CF=0 且 ZF=0。

同样的寄存器位串可以被程序当作有符号或无符号数，硬件减法结果相同，所选分支条件不同。`loop` 在 32 位计数约定下先把 ECX 减 1，再判断是否非零，它本身不等价于“修改 ZF 后的普通 dec 指令”；在分析后续标志位时要遵守具体指令的副作用定义。

4.10.2 用机器级控制流还原循环

```
mov ecx, 0
mov eax, 0
L_test:
    cmp ecx, 10
    jge L_done
    add eax, dword ptr [ebx+ecx*4]
    add ecx, 1
    jmp L_test
L_done:
```

若 EBX 指向 10 个 32 位整数且运算结果在目标类型范围内，这段代码将数组元素累加到 EAX。读程序时依次确定初始化、循环条件、循环体和更新。条件转移写在循环开头时可能一次也不执行；写在循环末尾并由循环体起步时通常至少执行一次，二者对应的高级语言结构不同。

4.11 函数调用：参数、返回地址和栈帧

采用课件中的 32 位、向低地址增长的栈，以及常见的 EBP 栈帧形式：调用者安排参数并执行 `call`；`call` 把下一条指令地址压栈，再跳到被调用函数。被调用者执行 `push ebp; mov ebp, esp`，保存旧帧基址并建立新帧；需要局部空间时再减少 ESP。

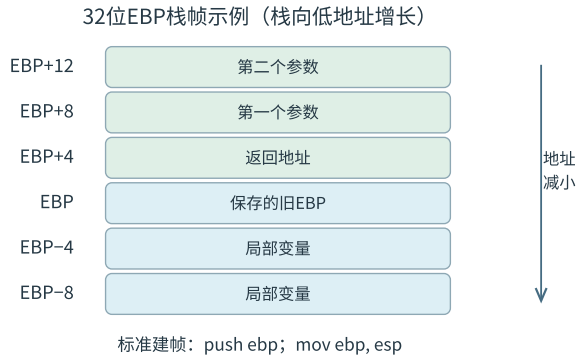


Figure 4.2: 32 位示例栈帧中，返回地址与参数位于 EBP 上方，局部变量位于下方

典型布局中，`[ebp]`为旧 EBP，`[ebp+4]`为返回地址，`[ebp+8]`、`[ebp+12]`为前两个栈参数，局部变量位于 `[ebp-4]`等位置。参数次序、哪些寄存器由谁保存、由谁回收参数空间，属于调用约定；这里按上述教学模型分析，不把某一布局当作全部编译器和优化级别的固定形式。

函数返回前常用 `mov esp, ebp; pop ebp` 恢复调用者栈帧，随后 `ret` 弹出返回地址到指令指针；常见整数返回值放在 EAX。递归调用每层拥有独立的返回地址和必要局部状态；全局变量则由各层共享。计算递归空间时应乘以最大同时存活的调用层数，而不是总调用次数。

4.11.1 计算与辨析自查

1. 16 位指令的三地址操作码为何最多分配 15 种后还能继续扩展？2. `lea eax,[ebx+4]`会读取地址 EBX+4 的内容吗？3. 有符号小于为什么不能只看 SF？4. 上述栈帧中返回地址位于哪里？

答案。1. 留出的一个 4 位前缀可再结合下一段 4 位形成 16 种长操作码。2. 不会，只计算地址值。3. 减法发生溢出时，结果符号与数学差的符号可能不同，须结合 OF。4. `[ebp+4]`，前提是已经执行标准建帧序列。

4.12 主动回忆自测

4.12.1 配套自测

1. 原创补充：沿用本章栈模型，初始 SP=1000H，连续 push 两个 4 B 值后，再 pop 一个值，SP 是多少？返回地址保存在链接寄存器时，是否每次调用都必然产生一次主存写？

答案。1. SP=0FFCH。否，是否写主存还取决于调用模型及是否需要溢出保存。

4.12.2 遮住答案复述知识关系

主动回忆·自拟

2. 自拟：说明零地址加法如何仍有两个源操作数；哪部分信息被隐含了？
3. 自拟：某 2B 转移指令从 2000H 开始，取完 PC 为 2002H，8 位

课程连接·按语义复查

地址计算、字节序和标志位分别检查，避免由指令名字直接猜数值结果。

补码位移 FAH。目标地址是多少?

4. 自拟：指令和操作数都占一个存储字，无 Cache 无预取，直接读操作数、一次主存间接读操作数、寄存器间接读操作数，各至少几次主存访问?
5. 自拟：原条件为 x<y，为何实现 if/else 时常先用 x>=y 跳到 else? 如果改成 unsigned，分支条件的解释要怎样改?
6. 自拟：返回地址存在链接寄存器时，递归调用为什么仍可能需要额外保存？调用后可以直接返回函数入口吗?

课程连接·按语义复查

地址计算、字节序和标志位分别检查，避免由指令名字直接猜数值结果。

4.12.3 参考答案

2. 源来自约定的栈顶两项，目的为约定栈顶；指令没有显式地址字段，不表示没有操作数。
3. FAH 为 -6，目标 $2002H - 6 = 1FFCH$。
4. 分别至少 2、3、2 次：都需一次取指；主存间接额外读一次地址，寄存器间接的地址已在寄存器。
5. 取原条件的否定，使满足原条件者顺序进入 then，其他跳到 else。unsigned 须采用无符号比较条件，不能直接复用有符号标志组合。
6. 再次调用会覆盖链接寄存器，须在需要返回的旧值被覆盖前保存。返回到调用点后继地址，而非函数入口；各层栈帧与返回位置必须匹配。